

جامعة نيويورك أبوظبي



NYU | ABU DHABI

Robotics

Fall 2025

Experiment : Lab 6

Due Date : October 28, 2025

Name: Demarce Williams (DSW9740)

Lab Report: Forward and Inverse Kinematics of Robotic Manipulators

1. Introduction

Forward kinematics (FK) is the mapping from joint coordinates, or robot configuration, to end-effector pose. In other words, the main purpose of FK is to answer the question: *“Given the joint states, what is the position and orientation of the end-effector?”*

Inverse kinematics (IK) performs the opposite operation. It answers: *“What joint states should be set to achieve a desired end-effector position and orientation?”*

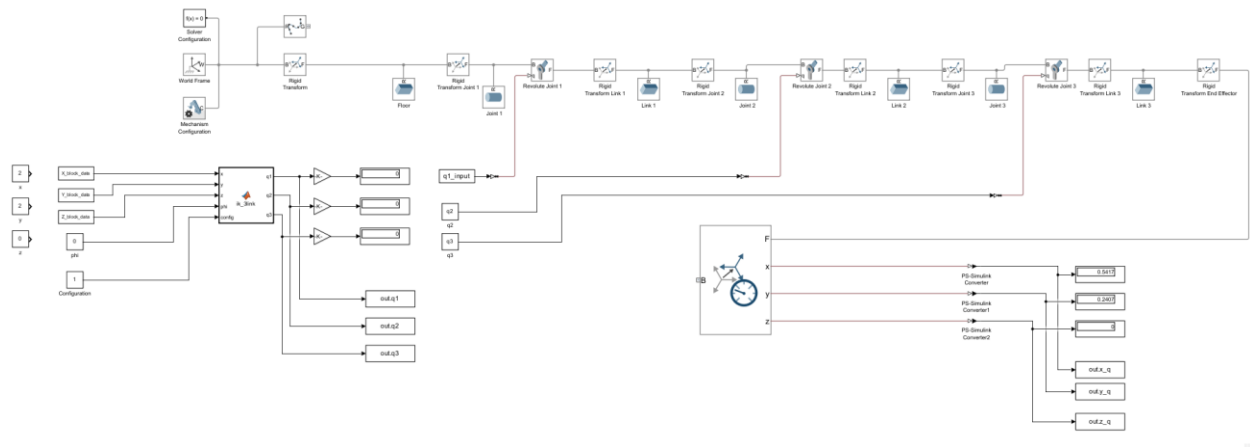
This lab explores FK and IK of two manipulators: a **3-link planar arm** and an **anthropomorphic arm**, implemented in MATLAB and Simscape. The lab also demonstrates path following and reachability checks.

Lab Goals:

1. Understand inverse kinematics of basic manipulators.
2. Practice Simscape modeling for robotic arms.

2. 3-Link Planar Arm

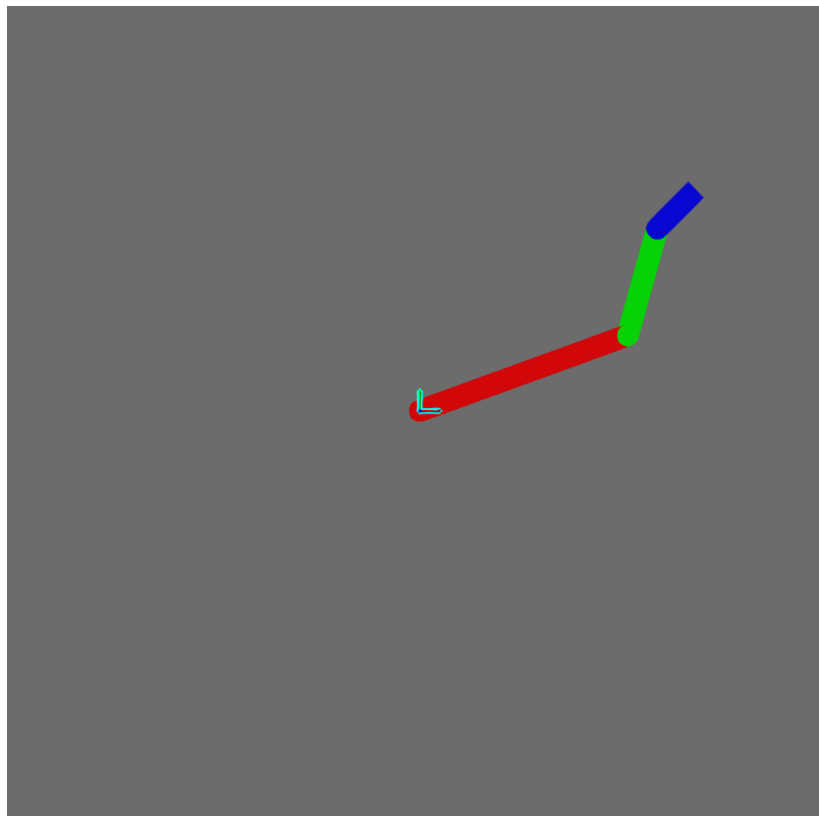
2.1. Simscape Model



2.2. Forward and Inverse Kinematics Implementation

The manipulator's kinematics are described using both forward and inverse approaches. Forward kinematics (FK) computes the end-effector position and orientation from given joint angles $[q_1, q_2, q_3]$, while inverse kinematics (IK) computes the joint angles needed to reach a desired end-effector pose $[x, y, \phi]$.

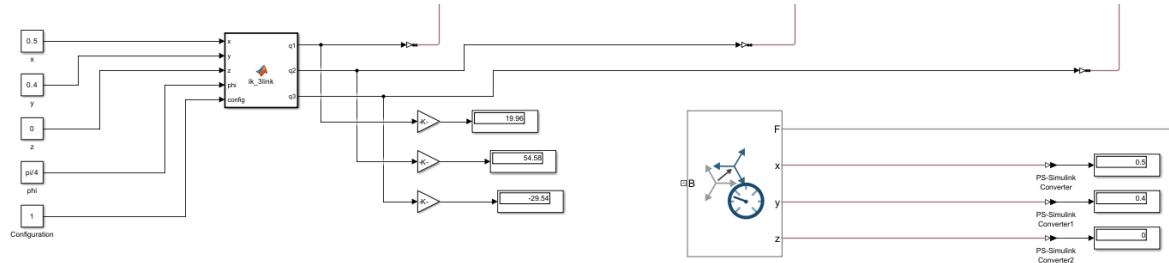
Figure 2.1: 3-Link Planar Arm Forward and Inverse Kinematics Visualization



This figure shows a sample manipulator configuration. The IK function computes joint angles for a specified end-effector pose, allowing verification by passing the IK output back through the FK function. The FK function calculates joint positions and end-effector pose from given joint angles.

Test Cases:

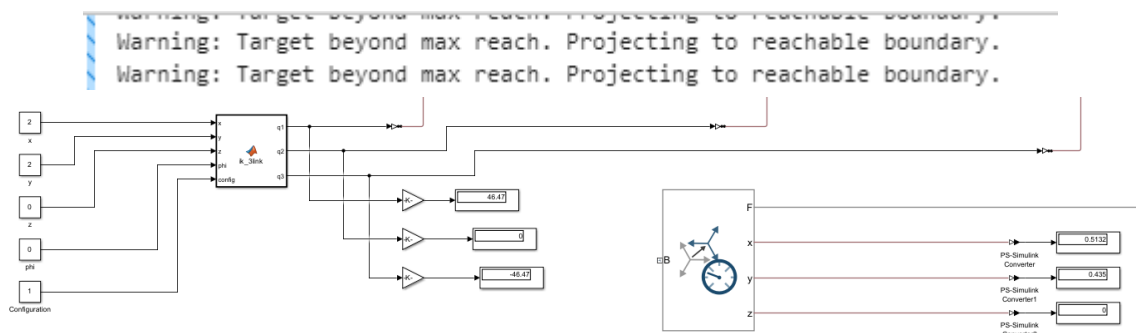
- End-effector at $[0.5, 0.4, \pi/4]$ with elbow down configuration
- Verification: Apply $[0.5, 0.4, \pi/4] \rightarrow \text{IK} \rightarrow [19.96, 54.58, -29.54] \rightarrow \text{FK} \rightarrow [0.5, 0.4, 0]$



2.3. Reachability Check

The reachability check ensures that any requested end-effector pose outside the manipulator's reachable workspace is projected onto the nearest feasible point. This prevents invalid joint angle calculations and ensures safe and realistic manipulator configurations.

Figure 3.1: Reachability Check of 3-Link Planar Arm



This figure demonstrates how end-effector points beyond the workspace boundary are projected to reachable positions. IK outputs are automatically corrected when targets are unreachable.

Test Cases for Reachability:

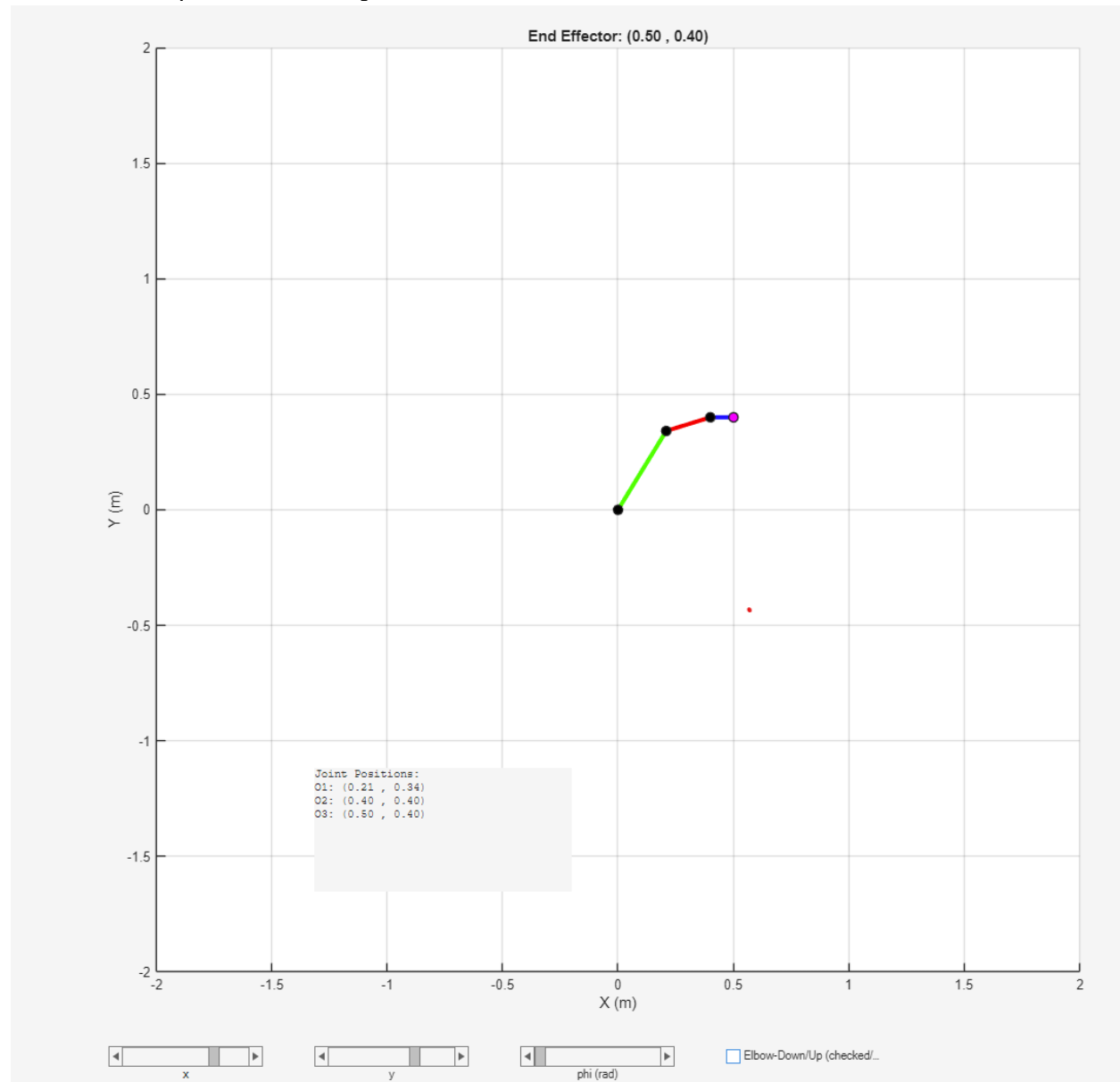
- Target at $[2.0, 2.0, 0]$ (beyond maximum reach)
- Verification: Apply $[2.0, 2.0, 0] \rightarrow \text{IK} \rightarrow [46.47, 0, -46.47] \rightarrow \text{FK} \rightarrow [0.513, 0.435, 0]$

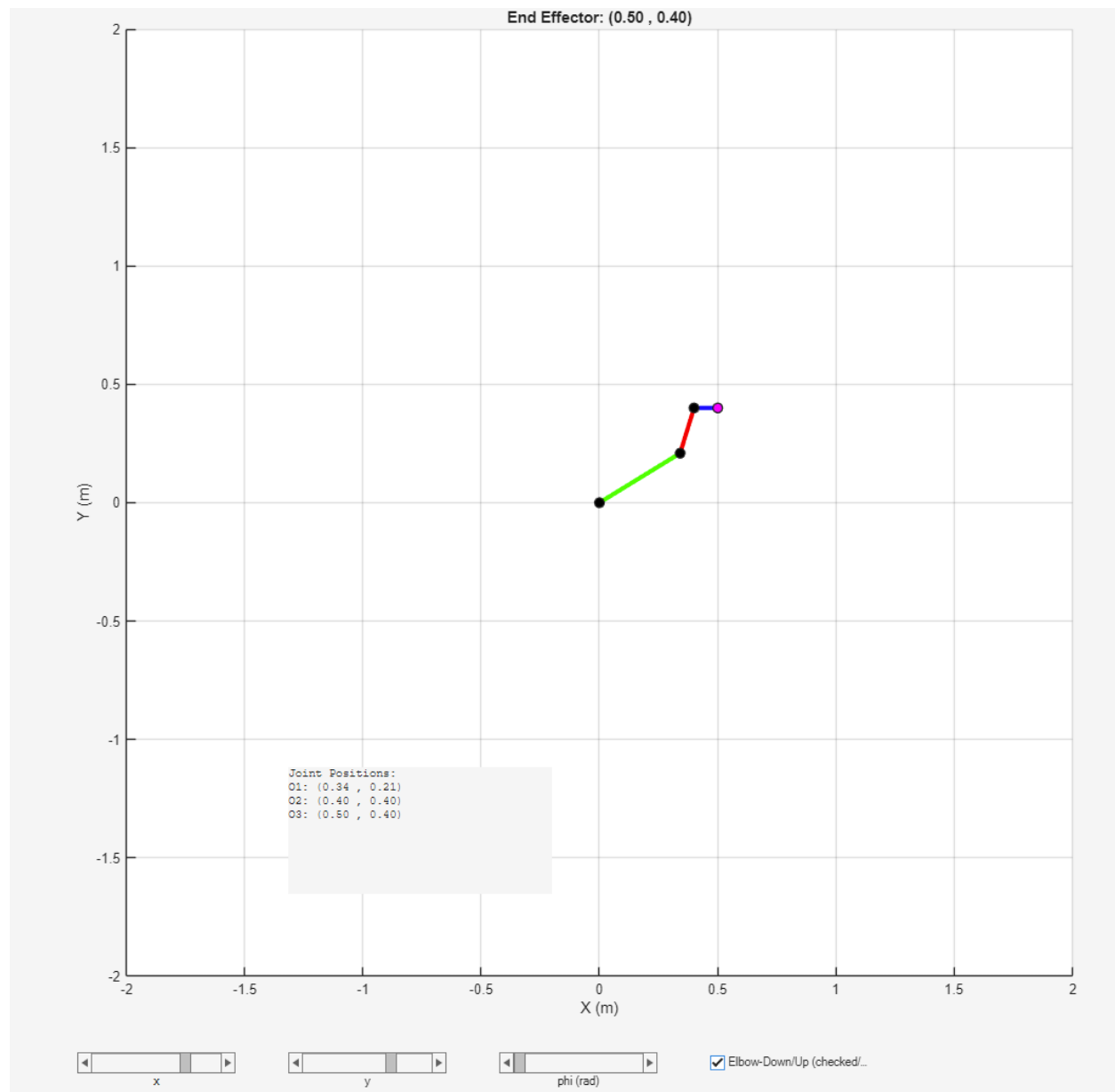
The output is not as expected since it is clamped and corrected to stay in the reachable workspace.

2.4. Live Script Interaction

A live script was implemented with sliders for x , y , and ϕ inputs, and a checkbox for elbow configuration. The script calls the IK function to compute joint angles, then FK to plot the manipulator in real-time.

Figure 2.4: MATLAB Live Script using Forward and Inverse Kinematics Functions and Sliders and Checkbox for Real-time Input Parameters

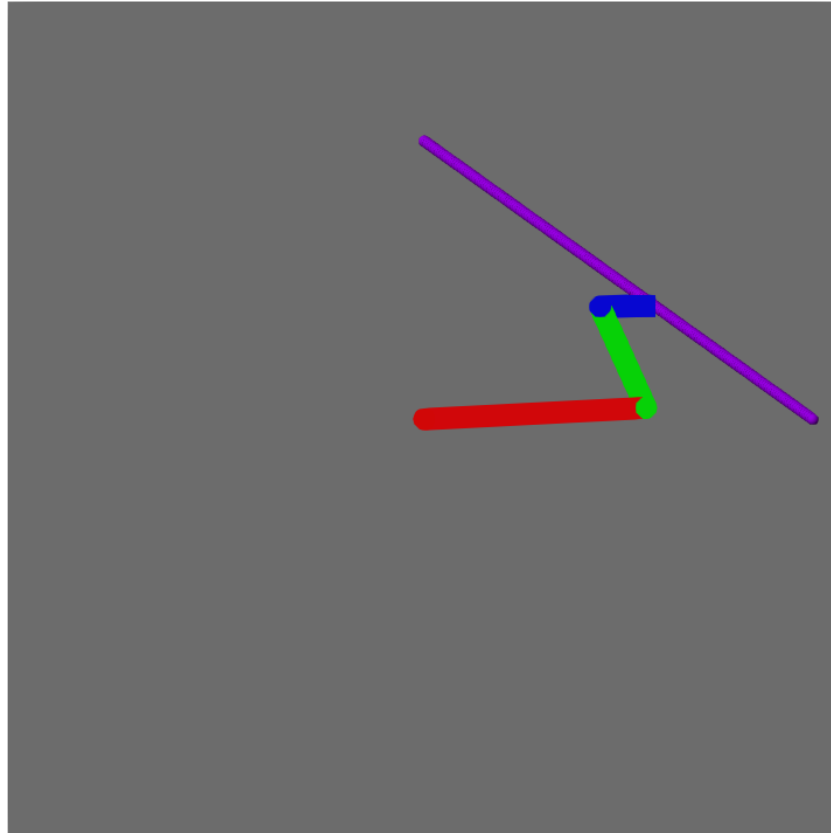




2.5. Path Following

A downward sloping straight line with x and y intercepts equal to 0.7, and 0.5 respectively in the workspace was generated with time vectors. To Workspace and From Workspace blocks were used to apply inputs and record joint states.

Figure 2.5: *3-Link Planar Arm Path Following in Mechanical Explorer*

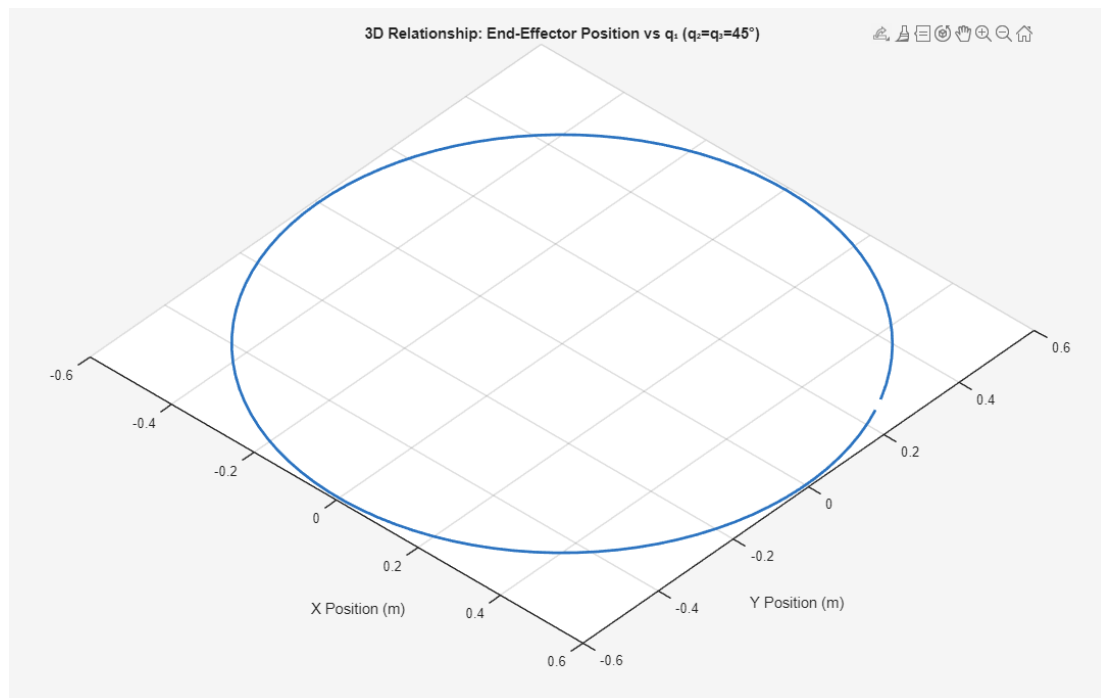
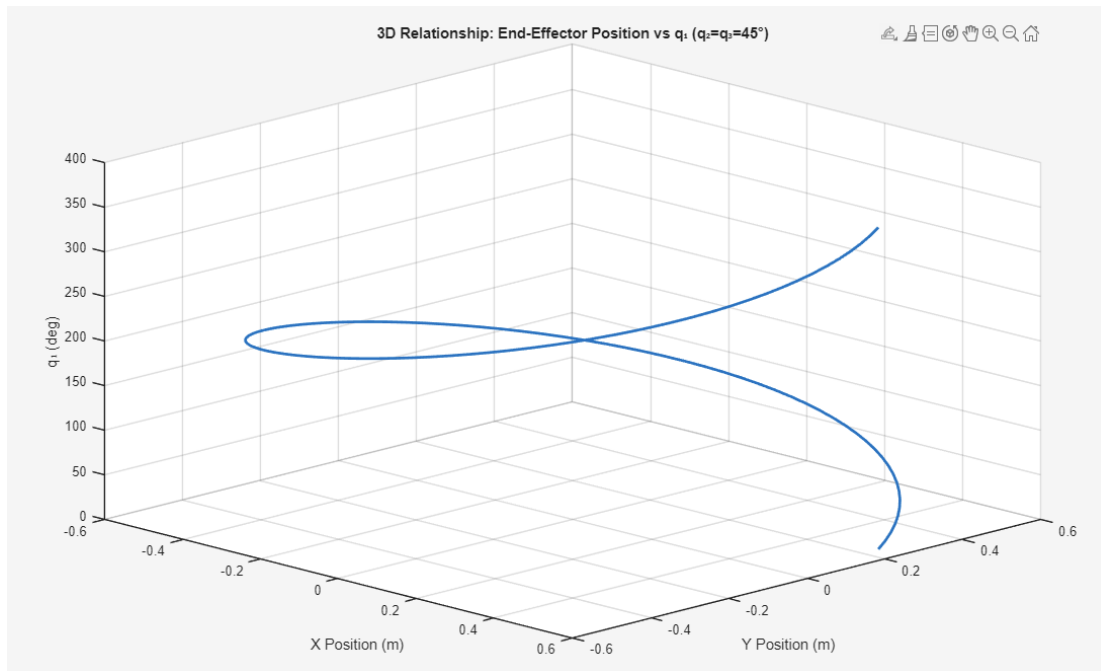


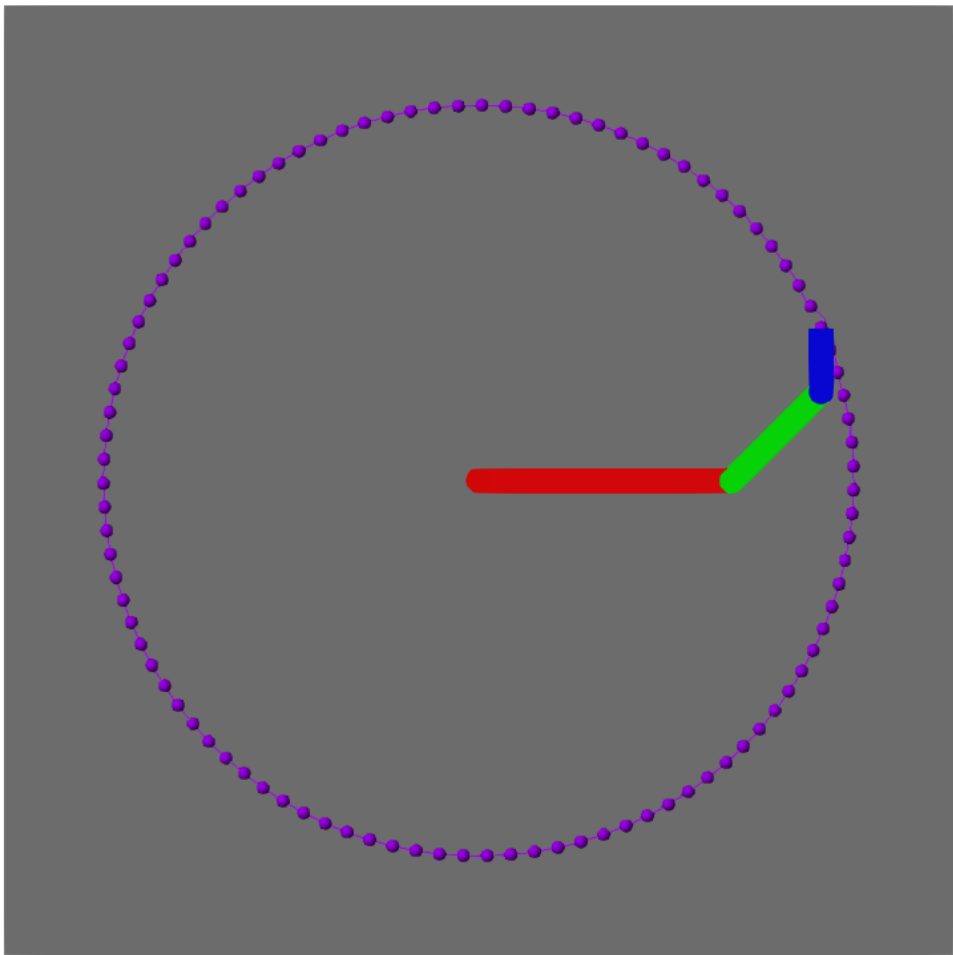
2.6. Effect of q_1 on End-Effector

With $q_2 = 45^\circ$ and $q_3 = 45^\circ$, varying q_1 changes the end-effector position.

Figure 2.6: *Effect of q_1 on End-Effector Position ($q_2=45^\circ$, $q_3=45^\circ$)*

This illustrates the relationship between joint 1 and the workspace position.

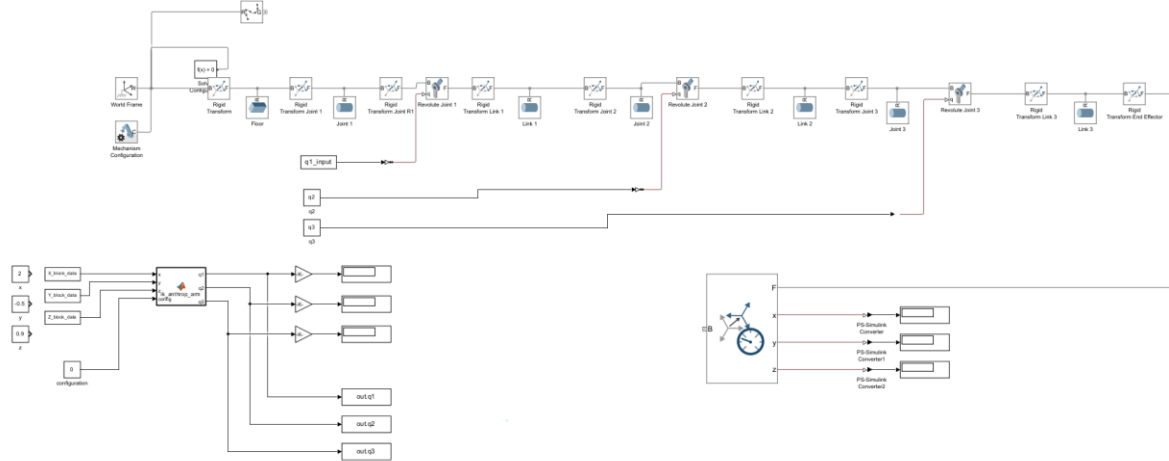




3. Anthropomorphic Arm

The same procedure as the 3-link planar arm was applied to the anthropomorphic manipulator, using `fk_anthrop.m` and `ik_anthrop.m`.

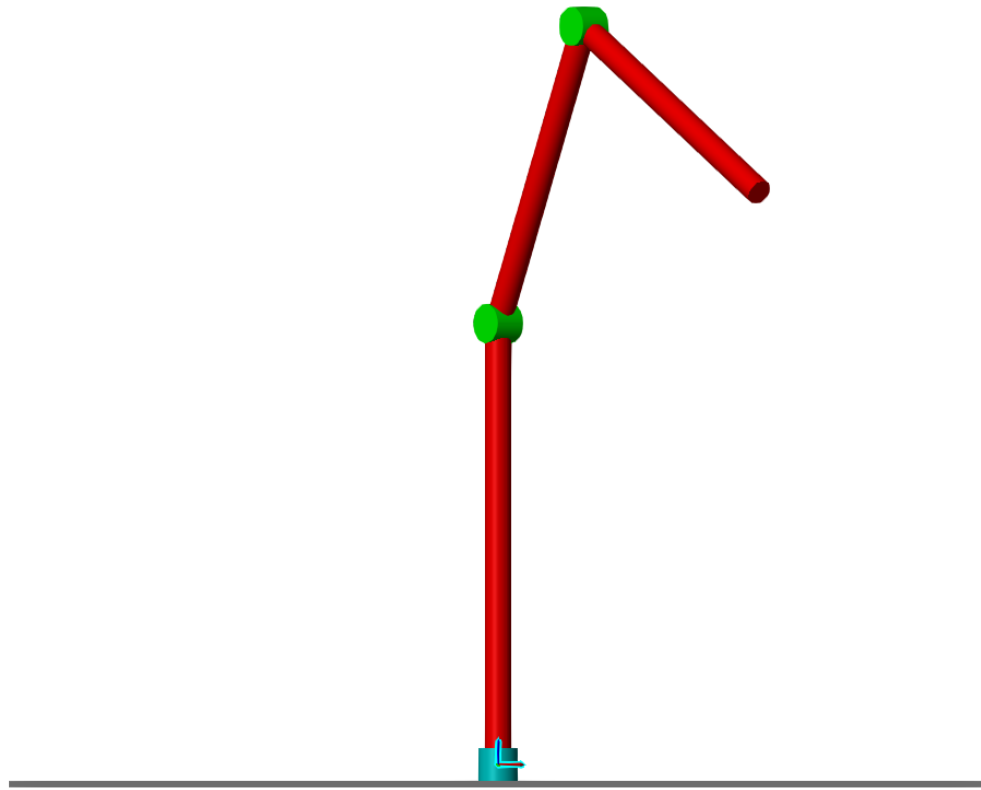
3.1 Simscape Model



3.2 . Forward and Inverse Kinematics Implementation

The manipulator's kinematics are described using both forward and inverse approaches. Forward kinematics (FK) computes the end-effector position and orientation from given joint angles $[q_1, q_2, q_3]$, while inverse kinematics (IK) computes the joint angles needed to reach a desired end-effector position $[x, y, z]$.

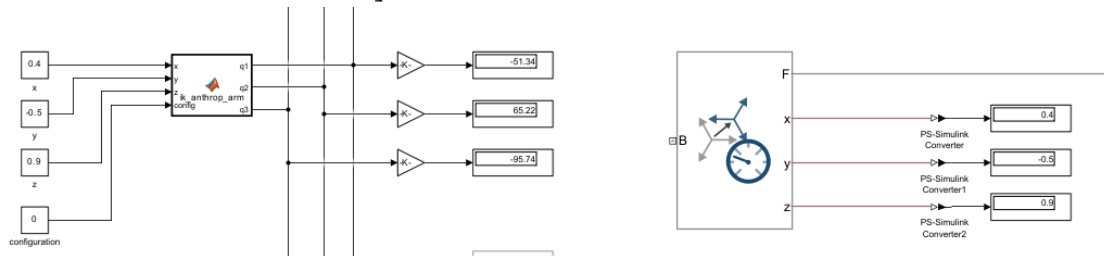
Figure 3.1: Anthropomorphic Arm Forward and Inverse Kinematics Visualization



This figure shows a sample manipulator configuration. The IK function computes joint angles for a specified end-effector pose, allowing verification by passing the IK output back through the FK function. The FK function calculates joint positions and end-effector pose from given joint angles.

Test Cases:

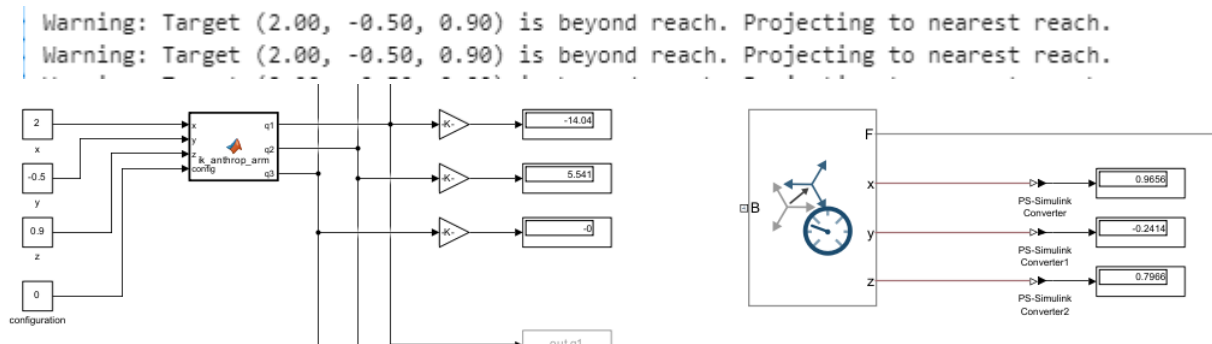
- End-effector at $[0.4, -0.5, 0.9]$ with elbow down configuration
- Verification: $[0.4, -0.5, 0.9] \rightarrow \text{IK} \rightarrow [-51.34, 65.22, -95.74] \rightarrow \text{FK} \rightarrow [0.4, -0.5, 0.9]$



3.3. Reachability Check

The reachability check ensures that any requested end-effector pose outside the manipulator's reachable workspace is projected onto the nearest feasible point. This prevents invalid joint angle calculations and ensures safe and realistic manipulator configurations.

Figure 3.3: Reachability Check Demonstration for Anthropomorphic Arm



The reachability check ensures that targets outside the workspace are projected to reachable boundaries.

Test Cases for Reachability:

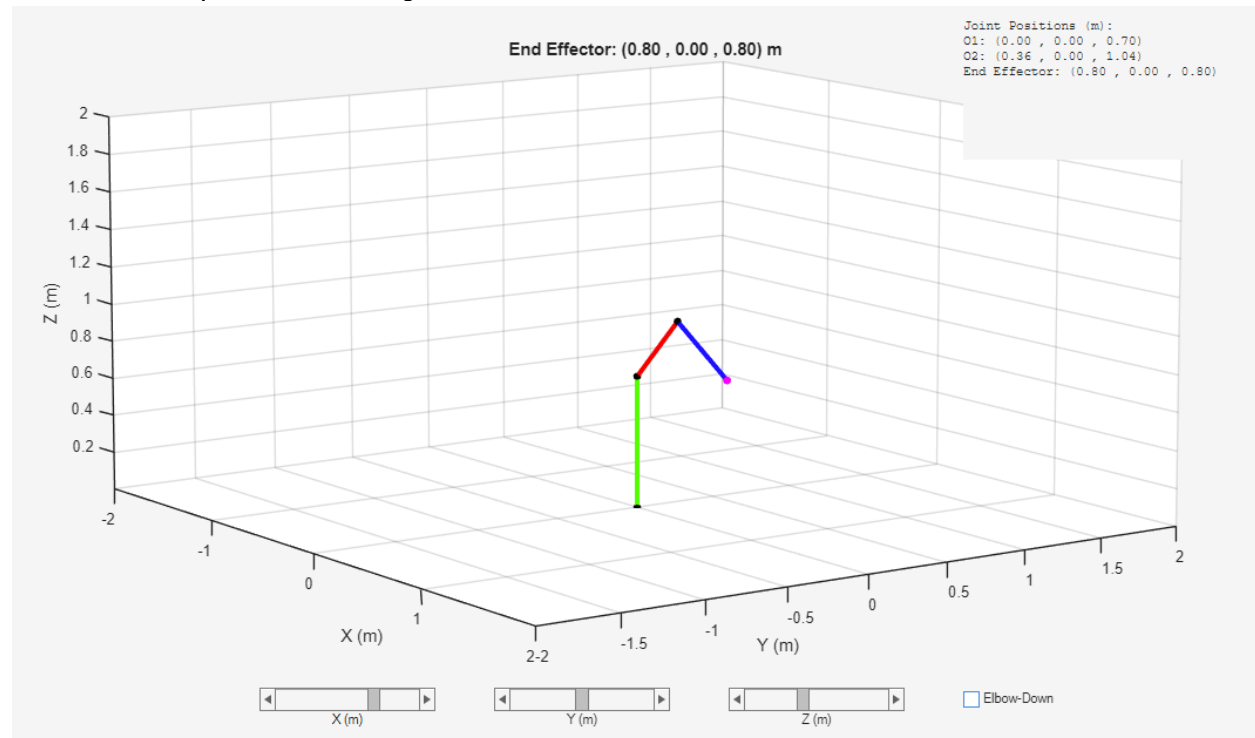
- Target at [2.0, -0.5, 0.9] (beyond maximum reach)
- Verification: Apply [2.0, -0.5, 0.9] $\rightarrow$ IK $\rightarrow$ [-14.04, 5.541, 0] $\rightarrow$ FK $\rightarrow$ [0.513, 0.435, 0]

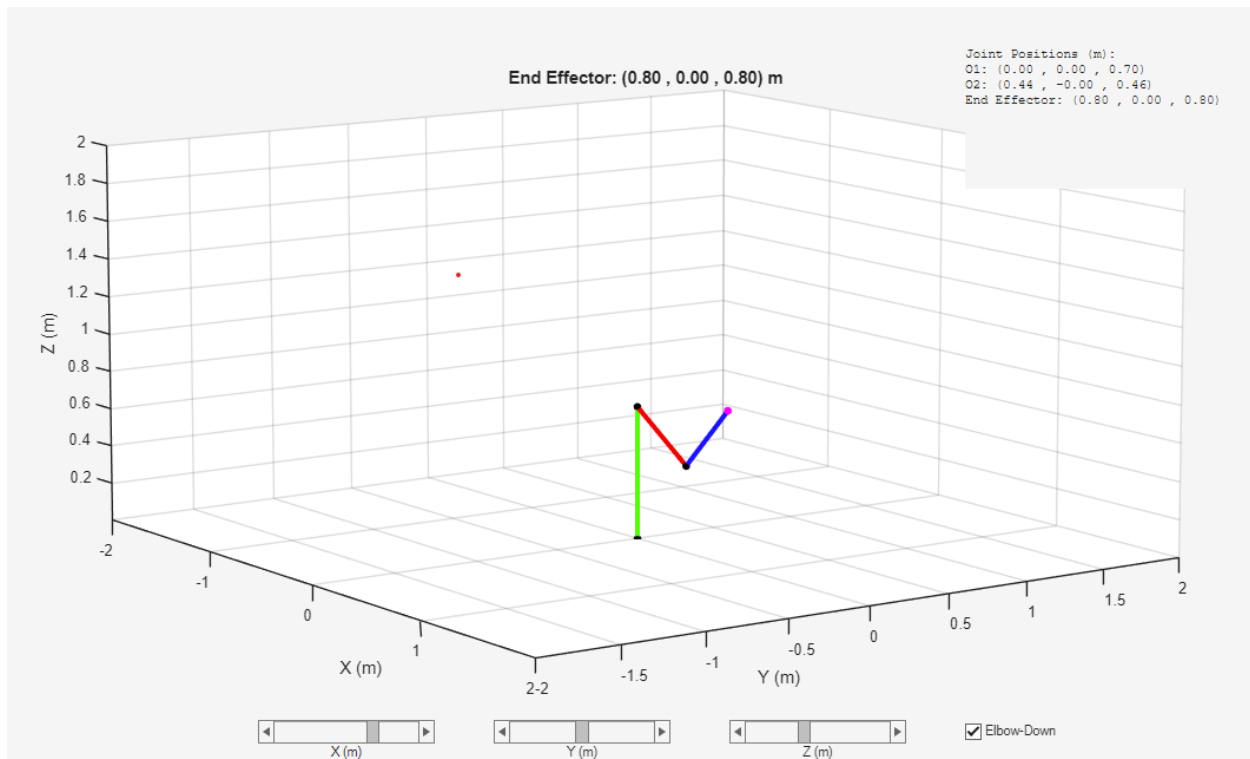
The output is not as expected since it is clamped and corrected to stay in the reachable workspace.

3.3 Live Script Interaction

A dashboard interface with sliders and checkboxes allows interactive control of the manipulator.

Figure 3.4: MATLAB Live Script using *Forward and Inverse Kinematics Functions and Sliders and Checkbox for Real-time Input Parameters*

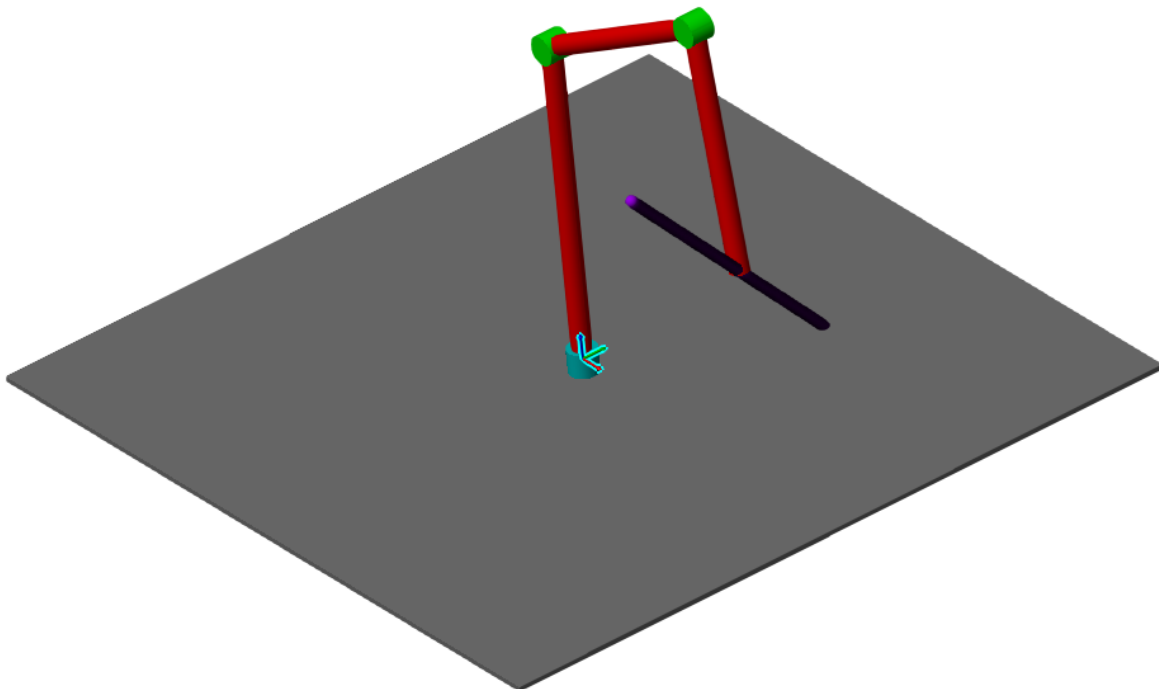




3.4 Path Following

The manipulator follows a straight-line trajectory along the X-axis using workspace inputs.

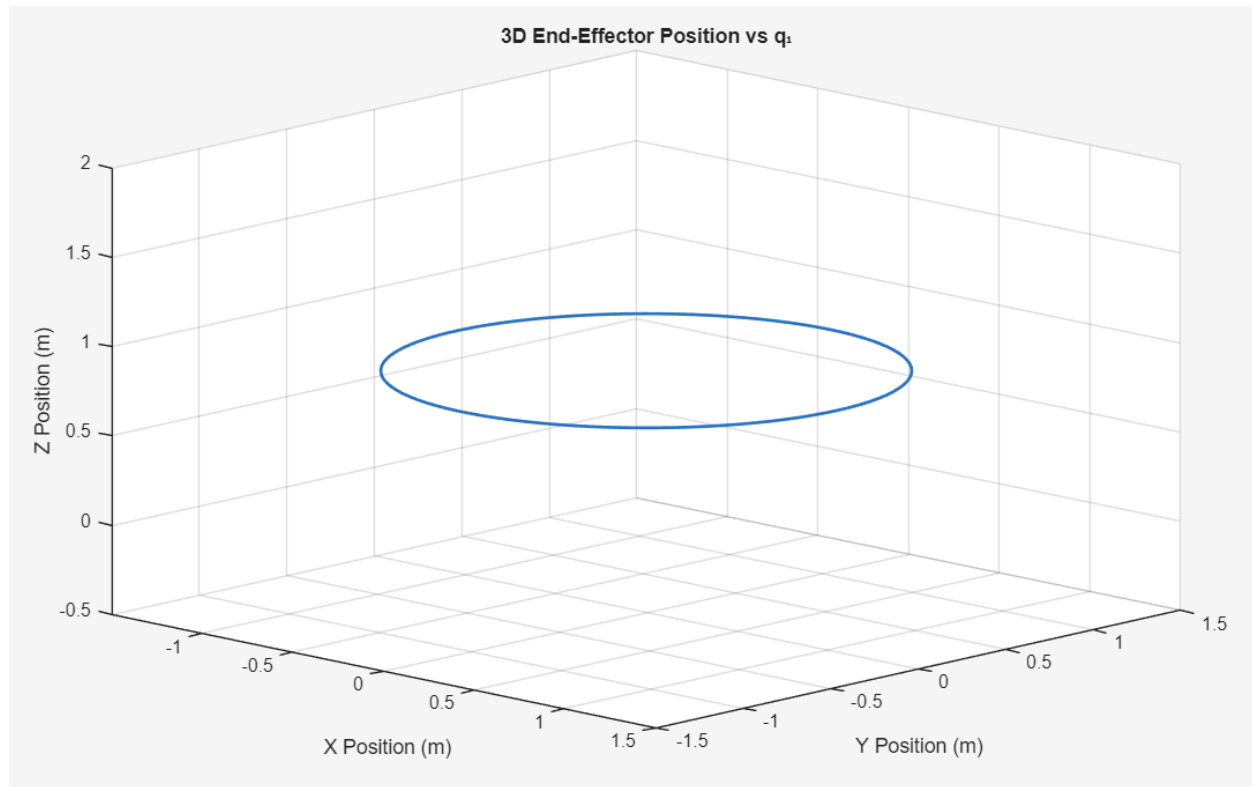
Figure 3.5: *Anthropomorphic Arm Path Following in Mechanical Explorer*

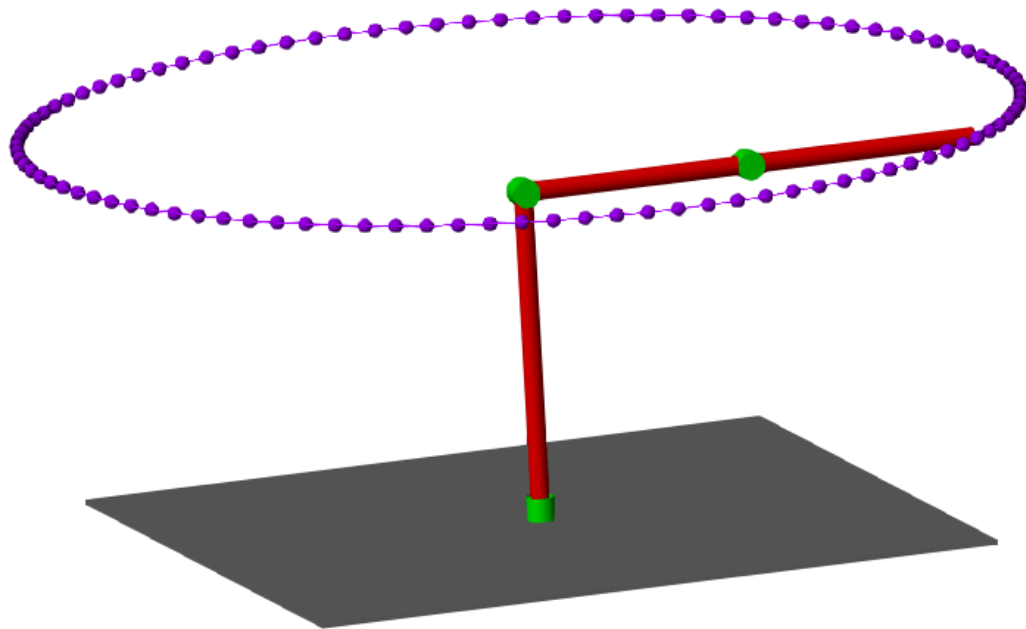
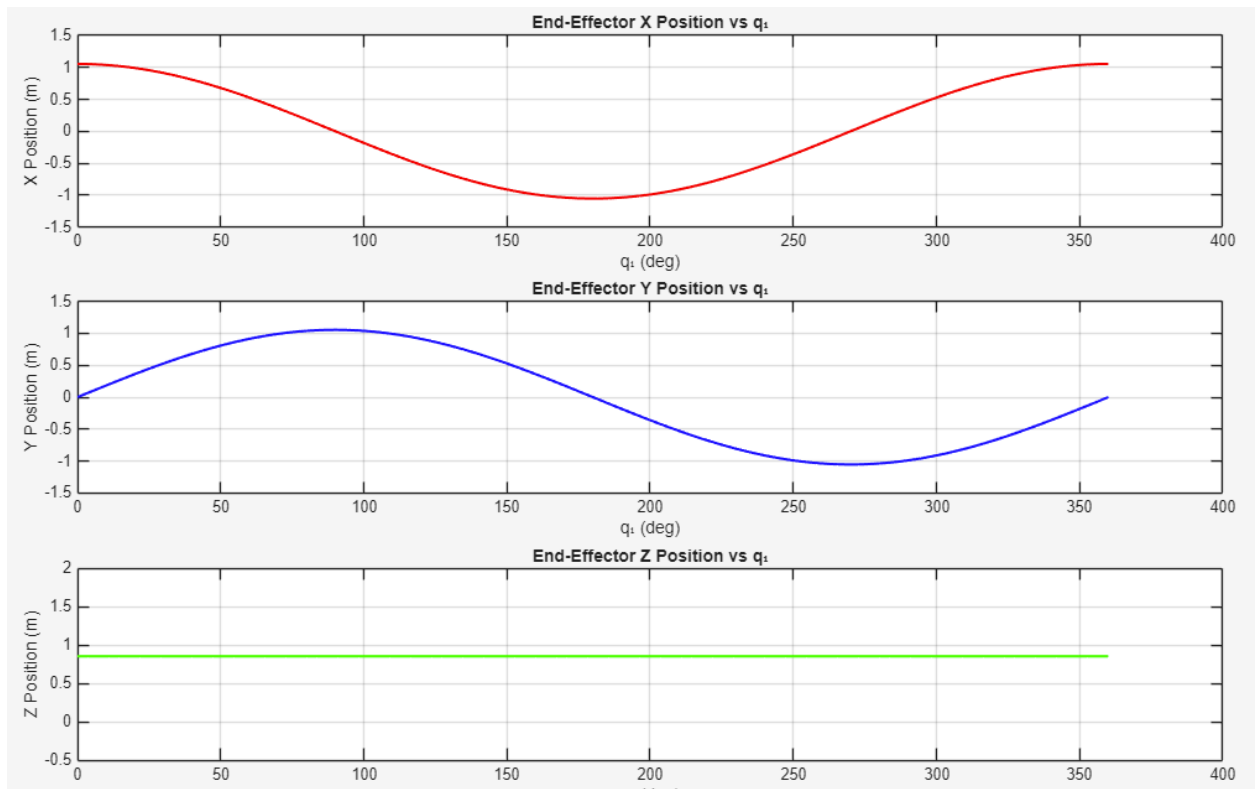


3.5 Effect of q_1 on End-Effector

With other joints fixed, q_1 changes the end-effector position.

Figure 3.6: *Effect of q_1 on End-Effector Position (Other Joints Fixed)*





4. Discussion

In this lab, we explored the kinematic modeling of robotic manipulators, both from a theoretical perspective and through simulation in MATLAB and Simscape. The focus was on two types of arms: a 3-link planar arm and an anthropomorphic arm. Through the use of Forward Kinematics (FK) and Inverse Kinematics (IK), along with reachability checks and trajectory following in Mechanical Explorer, several key insights were gained.

The 3-link planar arm is a simple manipulator with three degrees of freedom (DOF) that operates within a single plane. The key advantage of this type of manipulator is its geometric simplicity. Inverse kinematics for this arm can be solved directly using geometric methods, making it an excellent example for understanding the basics of kinematics. The relationship between the joint angles and the end-effector position is straightforward, and the joint configurations can be easily calculated.

In the context of the lab, we used a forward kinematics function to map the joint states to the position of the end-effector. This allowed us to simulate and visualize the arm's motion. Since the arm is planar, the forward kinematics involves basic trigonometric calculations, which we implemented in the MATLAB code.

On the other hand, the anthropomorphic arm is more complex, with multiple DOF and an articulated structure that mimics a human arm. This type of manipulator has greater flexibility, which is desirable for more complex tasks but also introduces new challenges.

In this lab, we implemented the inverse kinematics solution for the anthropomorphic arm, which involved not only the position of the end-effector but also the configuration of the arm (elbow-up or elbow-down). This required a careful consideration of the arm's reach and joint constraints.

One of the key challenges with higher-DOF arms, such as the anthropomorphic arm, is ensuring that the desired end-effector position is within the arm's reachable workspace. In the lab, we implemented reachability checks to ensure that the input end-effector position is valid. Without these checks, the manipulator could attempt to reach a point outside its workspace, resulting in errors or unrealistic behavior. The reachability check is essential to avoid trying to compute inverse kinematics for an unreachable point, which would lead to invalid or non-feasible joint configurations.

The lab provided an opportunity to test path following in Mechanical Explorer, where the manipulator was tasked with tracking a desired trajectory. The path-following functionality in Simulink's Mechanical Explorer block allowed us to visualize the manipulator's motion in 3D, confirming that the manipulator accurately followed the trajectory defined by the inputs. By varying the joint angles (such as q_1), we were able to observe how the end-effector position changed, providing useful feedback about the manipulator's performance in dynamic environments.

This feature was particularly useful when testing the anthropomorphic arm. By defining a path along the X-axis, we could visualize the arm's motion in response to changes in joint configurations. The simulation confirmed that the manipulator was capable of accurately tracking the desired trajectory, reinforcing the relationship between the joint angles and the end-effector position.

The live script interfaces, with interactive sliders for joint angles and end-effector positions, proved to be highly effective for testing both inverse and forward kinematics. The ability to adjust joint angles dynamically and immediately observe the changes in the manipulator's configuration in real-time facilitated a deeper understanding of how the inverse kinematics function works. This interactive approach made it easier to debug, visualize, and fine-tune the system.

By adjusting sliders for the X, Y, and Z coordinates of the end-effector and using the inverse kinematics function to compute the corresponding joint angles, we could visualize the manipulator in its new configuration. This hands-on approach allowed for an intuitive understanding of how the arm moves and how different configurations affect its reach and trajectory.

The Simscape model of the anthropomorphic arm further extended our understanding of kinematics. Using dashboard blocks for input and output, we could simulate the manipulator's behavior and save the joint states during the simulation. This model also allowed us to visualize the relationship between the joint angles (q_1 , q_2 , q_3) and the end-effector position, specifically with q_2 and q_3 fixed at 45° . This visualization gave insight into how different joint configurations influence the reach of the arm.

Additionally, by generating a trajectory for the arm to follow, we could observe how the manipulator responds to predefined paths and ensure that it stays within its workspace. The use of "To Workspace" blocks to apply inputs and save joint states further enhanced our ability to track the arm's movements over time, providing valuable data for analysis and verification.

5. Conclusion

In conclusion, this lab provided a comprehensive exploration of forward and inverse kinematics for both a simple 3-link planar arm and a more complex anthropomorphic arm. The key takeaways include the importance of workspace checks to ensure valid solutions for inverse kinematics, the effectiveness of Mechanical Explorer for path visualization, and the usefulness of interactive live scripts for testing and refining kinematic functions. By combining geometric methods with numerical techniques, we were able to model and simulate the behavior of both manipulators, providing valuable insights into robotic motion and control.

Appendix

1. MATLAB files (fk_3link.m, ik_3link.m, fk_anthrop.m, ik_anthrop.m, live scripts) are all attached.